

Noções muito básicas de complexidade

Helena Graziottin Ribeiro
hgrib@ucs.br

Complexidade

- **Como comparar o desempenho de algoritmos?**

Complexidade

- Costuma-se medir a eficiência de um algoritmo em termos de:
 - **tempo** de execução
 - ou **espaço** (ou memória) usado
- Para o **tempo**, podemos considerar o tempo absoluto de execução (em minutos, segundos, etc.)
 - Medir o tempo absoluto não é interessante porque **depende da máquina** em que foi executado

Complexidade

- Através das estimativas de desempenho pode-se determinar a **complexidade de um algoritmo**
- **Definição:** a complexidade de tempo de um algoritmo, ou simplesmente **complexidade**, é a quantidade de trabalho executada pelo algoritmo.

Complexidade

Determinação da complexidade de um algoritmo:

- é escolhida uma (ou mais) operação do algoritmo, chamada de **operação fundamental**:
 - **é uma operação chave** - por exemplo, para os algoritmos de ordenação, pode ser a operação de troca.
- A complexidade é baseada na **contagem do número de operações fundamentais** realizada pelo algoritmo durante sua execução.
 - **Exemplos**: quantidade de comparações ou trocas em um algoritmo de classificação (ordenação).

Complexidade

- Normalmente a complexidade é uma função do tamanho (ou da quantidade de itens) da entrada: n
- **Notação O**: função que estima o tempo de execução, considerando n entradas: $O(f(n))$
- **Exemplo**: se a complexidade de um algoritmo é definida por $O(n)$, isso significa que o tempo de busca para n elementos é proporcional ao número de entradas do algoritmo. Assim, se o número de entradas aumenta três vezes, o tempo de busca triplica.

Funções de complexidade

Busca-se obter algoritmos onde $f(n)$ seja a "menor" função possível:

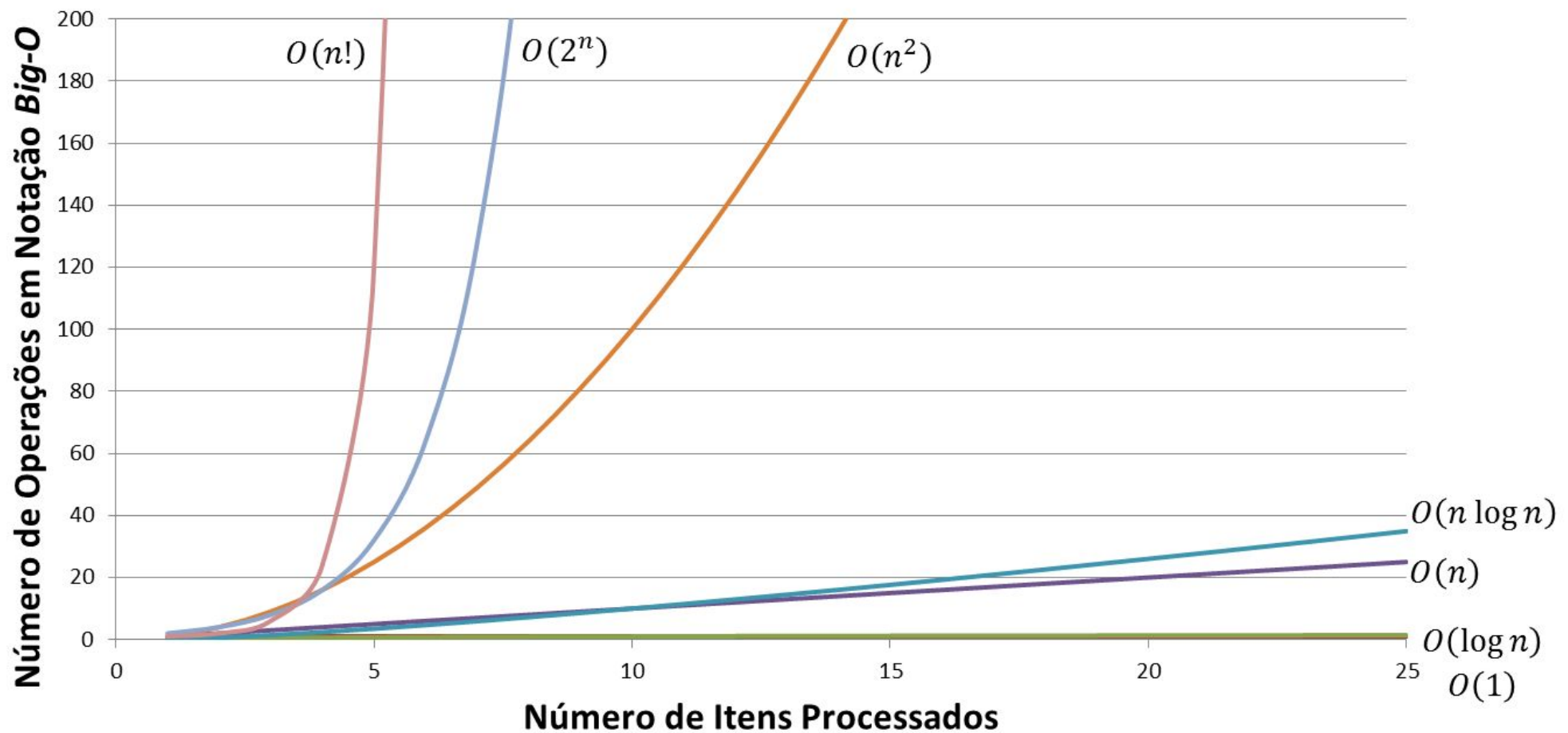
- $O(1)$ é melhor que
- $O(\log n)$ é melhor que
- $O(n)$ é melhor que
- $O(n \log n)$ é melhor que
- $O(n^2)$ é melhor que
- $O(2^n)$ é melhor que
- $O(3^n)$ etc...

Tratáveis em tempo
razoável
(polinomiais)

Intratáveis
(exponenciais)

Funções de complexidade

Ilustração das Complexidades Mais Comuns - Notação *Big-O*



Funções de complexidade

Execution times of a machine that executes 10^9 steps by second (~ 1 GHz), as a function of the algorithm cost and the size of input n :

Size	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n
10	3.322 ns	10 ns	33 ns	100 ns	1 μs	1 μs
20	4.322 ns	20 ns	86 ns	400 ns	8 μs	1 ms
30	4.907 ns	30 ns	147 ns	900 ns	27 μs	1 s
40	5.322 ns	40 ns	213 ns	2 μs	64 μs	18.3 min
50	5.644 ns	50 ns	282 ns	3 μs	125 μs	13 days
100	6.644 ns	100 ns	664 ns	10 μs	1 ms	$40 \cdot 10^{12}$ years
1000	10 ns	1 μs	10 μs	1 ms	1 s	
10000	13 ns	10 μs	133 μs	100 ms	16.7 min	
100000	17 ns	100 μs	2 ms	10 s	11.6 days	
1000000	20 ns	1 ms	20 ms	16.7 min	31.7 years	

Funções de complexidade

	10	100	10^3	10^4	10^5	10^6
$\log_2 n$	3	6	9	13	16	19
n	10	100	1000	10^4	10^5	10^6
$n \log_2 n$	30	664	9965	10^5	10^6	10^7
n^2	100	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	10^3	10^{30}	10^{300}	10^{3000}	10^{30000}	10^{300000}

Funções de complexidade

- O número de operações realizadas por um determinado algoritmo pode depender da organização dos dados de entrada
- Em geral nos interessa o **pior caso**, i.e., o maior número de operações usadas para qualquer entrada de tamanho n

Funções de complexidade

- Análises também podem ser feitas para o **melhor caso** e o **caso médio**.
 - para o caso médio, supõe-se uma certa organização da entrada:
 - Exemplo: Busca seqüencial de um dado elemento em um vetor armazenando **n** elementos de **forma aleatória**
 - **n** elementos em vetor ordenado?